Machine Learning Project Report

1 Introduction

This project studies the prediction of addiction levels from gaming-related features using both classification and regression tasks. We evaluate four methods: Dummy Classifier, k-Nearest Neighbors (KNN), Logistic Regression, and Linear Regression. The objective is to compare their performance and analyze their behavior in terms of generalization, overfitting, and computational efficiency.
In addition to the required methods, we include extended analysis such as runtime comparison and hyperparameter visualization.

2 Methodology

All features are normalized as: $x_{\text{norm}} = \frac{x-\mu}{\sigma}$ where μ and σ are computed on the training set. When $\sigma = 0$, it is replaced by 1 for numerical stability.
An 80/20 train-validation split is used during development.
The bias term is not added in preprocessing but handled internally by the relevant models.

2.1 Models

Dummy Classifier. Predicts the most frequent class and serves as a baseline.

KNN. Uses Euclidean distance. For classification, it predicts the majority class among the K nearest neighbors. For regression, it uses distance-weighted prediction:

$$\hat{y} = \frac{\sum_{i=1}^{K} w_i y_i}{\sum_{i=1}^{K} w_i}, \qquad w_i = \frac{1}{d_i + \epsilon} \tag{1}$$

Logistic Regression. A multi-class classifier trained with gradient descent on a softmax output.

Linear Regression. Uses the closed-form solution:

$$w = (X^T X)^{-1} X^T y$$

3 Results

Table 1: Classification results

Model	Train Acc	Test Acc	Train F1	Test F1
Dummy	68.75%	69.69%	0.272	0.274
KNN	100.00%	76.25%	1.000	0.584
Logistic	83.44%	82.19%	0.523	0.513

Table 2: Regression results (MSE)

Model	Train MSE	Test MSE
KNN	0.000	3.372
Linear Reg.	0.998	0.941

3.1 Classification

KNN achieves perfect training accuracy, indicating strong overfitting, especially for small values of K. Logistic Regression achieves the best test performance and provides the best generalization on unseen data.

3.2 Regression

Linear Regression significantly outperforms KNN on the test set. The near-zero training error of KNN again reflects overfitting, whereas Linear Regression generalizes much better.

3.3 Hyperparameter Analysis

KNN performance was evaluated for

$$K \in \{1, 3, 5, 7, 9, 11, 15\}.$$

Small values of K lead to overfitting, while larger values increase bias. A good trade-off is obtained around $K \approx 5$.

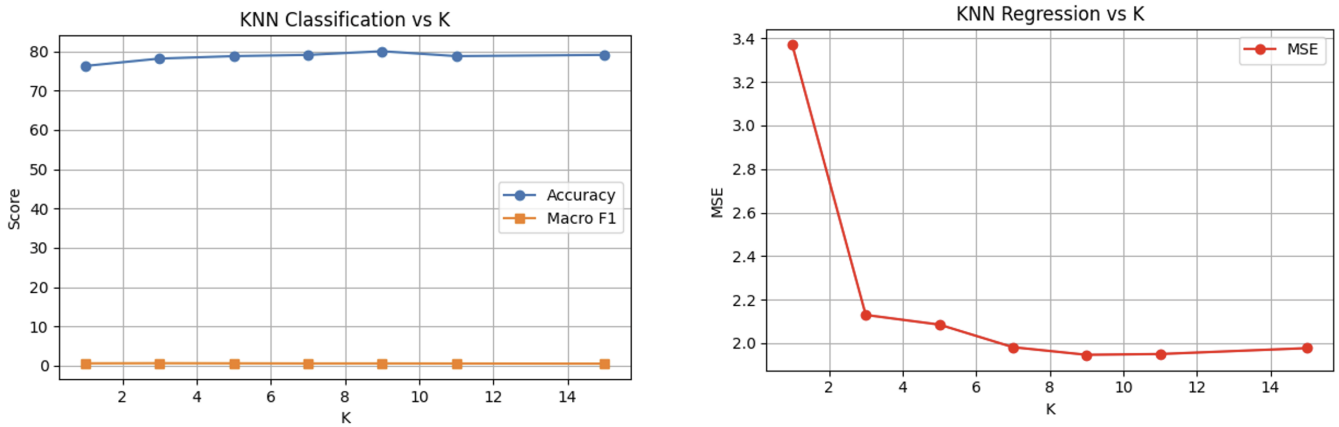

Figure 1: KNN performance as a function of K: classification (left) and regression (right).

The classification metrics remain relatively stable for moderate values of K, while regression exhibits a clear U-shaped error curve, illustrating the bias-variance trade-off.

3.4 Runtime Analysis

Table 3: Runtime comparison (seconds)

Model	Fit Time	Predict Time
Dummy	0.0001	0.0000
KNN	0.16	0.04
Logistic Reg	0.025	0.0002
Linear Reg	0.001	0.00004

KNN has negligible training time but high prediction complexity due to distance computations. Parametric models such as Logistic and Linear Regression are significantly more efficient at inference.

4 Conclusion

Logistic Regression provides the best classification performance, while Linear Regression is the most effective for regression tasks. KNN is flexible and simple to implement but suffers from overfitting at low K and high computational cost during prediction. These results highlight the importance of selecting appropriate models depending on the task and the structure of the data.

5 How to Run

The program can be executed using the template:

```
python main.py --method <method> --task <task> [options]
```

where `<method>` is one of `dummy_classifier`, `knn`, `logistic_regression`, or `linear_regression`, and `<task>` is either `classification` or `regression`.
A convenience option is also available to run all methods at once:

```
python main.py --method all
```